

Fused MoE GEMM 算子优化技术方案

XH-202608 基于国产软件栈大模型推理前沿算子优化 (TileLang 赛题)

评测账号: muxi2026C1050

最终成绩: XPU-OJ Accepted 80.33 (2026-09-05, submissionId 139770; 16 031 μ s 历史最快)

问题定义与评测

DeepSeek 风格 MoE pre-routed fused expert GEMM

$$\text{out} = (\text{SiLU}(x \cdot W_g) \odot (x \cdot W_u)) \cdot W_d \odot \text{routed}$$

三个正式用例 (fp16) :

case1 E=16, hidden=2048, inter=8192, valid=2272

case2 E=32, hidden=7168, inter=2048, valid=4544

case3 E=64, hidden=7168, inter=2048, valid=9088

计分: 每用例 $100 \cdot s / (1 + s)$, $s = \text{基线用时} / \text{内核用时}$; 三用例取均值

合规红线: 仅 TileLang; 禁 PyTorch 计算 / import_source / call_extern / 异步 / 跨调用缓存

硬件环境与核心挑战

MetaX C500: 64GB HBM; 实测 DRAM 峰值 ≈ 1.44 TB/s; L2 ≈ 8 MB; warp=64 线程

挑战 1: 无 cp.async 异步拷贝指令 —— 传统 double-buffer 流水线不可用

挑战 2: 评测机速度存在 $\pm 50\%$ 档位波动 —— 结论必须同窗口对照

挑战 3: 三用例形状差异大 (inter 8192 vs 2048; E 16 vs 64) —— 需要 per-shape 策略

方法论: 本地 judge 同构 harness + 32 组随机 seed 认证 + 同窗 OJ 对照

总体架构：两阶段 Fused 流水

Stage1 (fused Gate+Up) : 一次 A 块加载, 先后完成 gate / up 两次 MMA

$\text{gate_acc} += A \cdot W_g$; $\text{up_acc} += A \cdot W_u$ (fp32 累加)

epilogue 融合 SiLU: $\text{up_logits} = \text{SiLU}(\text{gate_acc}) \odot \text{up_acc}$ (仅有效行)

Stage2 (Down) : $\text{up_logits} \cdot W_d$, 逐行乘 routed, padding 行清零

tile 配置: $\text{bt}=128$ / $\text{bk}=64$ / $\text{bn}=128$, threads=256, MMA Square 策略

网格: $\text{Stage1} = \text{blocks_m} \times \text{inter}/128$; $\text{Stage2} = \text{blocks_m} \times \text{hidden}/128$

关键技术 ①：A 操作数 shared 化 (3.2× 吞吐)

问题：A (token 块) 放 fragment 时, LDS→寄存器分散搬运成为 MMA 供给瓶颈

方案：A 改为 alloc_shared, MMA 直接以 shared 为操作数

纯 GEMM 吞吐：26.8 → 87 TFLOPS (3.2×)

附加收益：gate/up 两次 MMA 复用同一份 A 块, A 全局加载次数减半

权重加载使用 coalesced_width=8 (16B/线程) 访存合并

关键技术 ②：无异步指令的 load/MMA 重叠

约束：赛题禁用异步与流水线（且 MACA 无 cp.async）

方案：同步寄存器预取 —— up 权重块提前加载进 fragment 寄存器

gate MMA 执行期间，up 的全局加载由硬件记分板自然完成

gate MMA 结束后，仅做寄存器→shared 短途搬运，随即执行 up MMA

效果：hidden=7168 档正收益；case1 分派全开后 -6.8%

合规性：纯同步代码，无任何 async/流水线 API

关键技术 ③：pass 开关组合 + per-shape 分派

tl.enable_fast_math (4 个 kernel) : -0.5~1%, 三 case 一致正收益

tl.enable_lower_ldgstg_predicated (stage1_prefetch / stage2_fast) : 谓词化加载 lowering

tl.disable_safe_memory_legalize + disable_vectorize_256 (stage2_fast) : 逐字节等价对拍后启用

per-shape 分派: hidden $\geq$ 7000 与 case1 的 K 循环长度/访存结构差异大, 统一启用增强 kernel

(分派全开) : case1 -6.8% / case2 -2.9% / case3 -3.7%

技术路线演进 (OJ 实测)

v6	官方模板调参基线	72.67
v138	A-shared + 权重 cw4 + column swizzle	75.67
v226	Stage1 MMA warp 策略 Square	76
v278	权重 copy coalesced_width 4→8	76.33
v282/v293	Stage2 column swizzle (两连 Accepted)	76.67
v404	Stage2 Down next-K 同步预取	78.00
v432	+fast_math +ldgstg_predicated +分派全开	78.67
v469	Stage2 手写同步调度 + swizzle 布局 vec4	79.33
v478	Stage1 双侧 vec4 布局 + cw 匹配 + panel2/kpack2	79.67
v718/v719/v720	E64 terminal-K + 双B emitter 扩展	80.33

结果与正确性

OJ: Accepted 79.33 (submissionId 137411, 2026-09-03)

OJ 三点: 总 16 031 μ s (历史最快) ; SPJ ratio 0.814/0.801/0.802

本地 C500: 2.789 / 5.790 / 9.137 ms (warmup10 / iters100)

功能测试: 7 个形状全通过 (含单 expert、valid=127 非整块、hidden=128 边界)

正确性: 对官方 fp32 参考语义逐元素对拍 bad=0; 32 组随机 seed 零失败

负结果亦全部记录: Pipelined $\geq$ 2 / CUDA 流 / M256 / 量化 / 手写 MFMA 等均实证不可行

仅使用 TileLang 语言原语与官方编译 pass 开关

T.Kernel / T.copy / T.gemm / T.Parallel / T.Pipelined(num_stages=1, 与官方模板一致)

无 PyTorch 计算算子 (torch 仅用于 host 侧工作区分配, 与官方模板一致)

无 import_source / call_extern / 外部设备库 (mcTlass 等)

无异步、无流水线语义 (v469 已将全部 T.Pipelined 移除)

无跨调用结果缓存 (仅 JIT 编译缓存与每次调用重填的 scratch 缓冲)